

Normalización del modelo de datos implementado

Plataforma de Tutorías entre Estudiantes · demostración del 2 de octubre de 2026

El sistema tiene cuatro servicios, cada uno con su propia base PostgreSQL, y ninguna clave foránea cruza de una base a otra (AD-08). En total son 21 tablas, y la normalización se revisa dentro de cada base, que es donde vive cada esquema relacional.

Base	Tablas
identity_db	usuario, token_verificacion, refresh_token, aviso_correo
catalog_db	carrera, malla, asignatura, malla_asignatura, prerequisito, sede, importacion_catalogo, importacion_error
tutoring_db	usuario_ref, asignatura_ref, perfil_tutor, bloque_disponibilidad, reserva, evaluacion, ejecucion_tarea
payment_db	pago, intento_pago

Las tres formas normales, en una línea cada una

- 1FN: cada celda guarda un solo valor y no hay grupos repetidos dentro de una fila.
- 2FN: además de 1FN, ningún atributo depende solo de una parte de una clave compuesta.
- 3FN: además de 2FN, ningún atributo que no es clave depende de otro atributo que tampoco lo es (no hay dependencias transitivas).

Todas las tablas tienen una clave primaria de una sola columna (uuid o bigint). Por eso la 2FN se revisa contra las claves candidatas compuestas, que en este modelo son cuatro:

Tabla	Clave candidata	Qué depende de ella
malla	(carrera_id, version)	vigente_desde y vigente_hasta
malla_asignatura	(malla_id, asignatura_id)	semestre, porque la misma asignatura puede ir en semestres distintos según la malla
perfil_tutor	(usuario_id, asignatura_id)	tarifa, descripcion y activo, porque cada tutor fija la tarifa de cada asignatura
prerequisito	(malla_asignatura_id, requiere_id)	nada: es una asociación pura

Lo que la revisión del 30-09 encontró y corrigió

El anexo A del Pilar 2 tenía tres tablas fuera de forma normal. Las tres se corrigieron en la implementación.

1. `importacion_catalogo` no cumplía la 1FN. La columna `errores_jsonb` guardaba en una sola celda la lista de filas rechazadas de una carga, y una lista en una celda es un grupo repetido. Ahora cada error es una fila de la tabla hija `importacion_error` (`importacion_id`, `fila`, `mensaje`), que se puede filtrar y contar con SQL como cualquier otra tabla.

2. `usuario_ref` no cumplía la 3FN. Tenía `carrera_nombre` y `sede_nombre`, pero esos nombres dependen de `carrera_id` y de `sede_id`, no del usuario. Es una dependencia transitiva: `usuario` → `carrera_id` → `carrera_nombre`. Si cien estudiantes fueran de la misma carrera, su nombre quedaría escrito cien veces en `tutoring_db`, y cambiarlo obligaría a tocar cien filas. Se quitaron las dos columnas; cuando haga falta mostrar el nombre, se pide a Catálogo.
3. `reserva` no cumplía la 3FN. Tenía `tutor_usuario_id`, pero el tutor de una reserva es el dueño del bloque reservado, así que se deduce de `bloque_id`: `reserva` → `bloque_id` → `tutor`. Se quitó la columna. La regla que usaba esa columna, «nadie reserva su propio bloque» (`ck_reserva_no_auto`, RN-03), sigue garantizada por la base: un disparador (`trg_reserva_no_auto`) compara al solicitante con el dueño del bloque y rechaza la fila con el mismo nombre de restricción. La lista de reservas de un tutor se obtiene uniendo `reserva` con `bloque_disponibilidad`.

Tabla por tabla

identity_db

Tabla	1FN	2FN	3FN	Comentario
usuario	Sí	Sí	Sí	El correo es clave candidata (único por <code>lower(email)</code>). Carrera y sede se guardan como identificador, sin nombre.
token_verificacion	Sí	Sí	Sí	<code>expira_en</code> es un hecho del token, porque la vigencia puede cambiar y no debe cambiar la de los tokens ya emitidos.
refresh_token	Sí	Sí	Sí	Igual que el anterior.
aviso_correo	Sí	Sí	Sí	<code>asunto</code> guarda lo que se envió de verdad, que puede incluir el nombre de la asignatura. No se deduce de la plantilla.

catalog_db

Tabla	1FN	2FN	3FN	Comentario
carrera	Sí	Sí	Sí	escuela es una etiqueta de la carrera. Sería una tabla si la escuela tuviera atributos propios.
mall	Sí	Sí	Sí	Separar malla de carrera permite cambiar el plan sin reescribir la historia.
asignatura	Sí	Sí	Sí	La sigla es clave candidata. Es transversal: una sola fila aunque la usen varias carreras.
mall_asignatura	Sí	Sí	Sí	semestre depende de la malla y de la asignatura juntas.
prerrequisito	Sí	Sí	Sí	Solo relaciona dos filas de malla_asignatura.
sede	Sí	Sí	Sí	Antes la sede era texto libre en el perfil (ajuste A-07).
importacion_catalogo	Sí (corregida)	Sí	Sí	Los contadores son el resumen de cada ejecución. Una fila omitida puede no tener error (por ejemplo, un duplicado).
importacion_error	Sí	Sí	Sí	Nueva.

tutoring_db

Tabla	1FN	2FN	3FN	Comentario
usuario_ref	Sí	Sí	Sí (corregida)	Ver la sección de copias de lectura.
asignatura_ref	Sí	Sí	Sí	Copia de lectura de sigla y nombre.
perfil_tutor	Sí	Sí	Sí	
bloque_disponibilidad	Sí	Sí	Sí	estado es un dato del bloque. ELIMINADO es el borrado lógico de un bloque con historial.

Tabla	1FN	2FN	3FN	Comentario
reserva	Sí	Sí	Sí (corregida)	monto es la tarifa congelada al momento de solicitar (A-04). No depende de perfil_tutor, porque la tarifa puede cambiar después sin tocar reservas ya hechas.
evaluacion	Sí	Sí	Sí	Una por reserva: reserva_id es clave candidata.
ejecucion_tarea	Sí	Sí	Sí	Registro de cada ejecución del programador.

payment_db

Tabla	1FN	2FN	3FN	Comentario
pago	Sí	Sí	Sí	Un pago por reserva: reserva_id es clave candidata. monto es lo que se cobró.
intento_pago	Sí	Sí	Sí	Cada reintento o reembolso es una fila (A-10). Por eso pago no pierde el historial.

Por qué las copias de lectura no rompen la normalización

usuario_ref, asignatura_ref y reserva.estado_pago repiten datos que son de otro servicio: el nombre del usuario viene de Identidad, la sigla de Catálogo y el estado del pago de Pagos. La normalización se mide dentro de un esquema, y dentro de tutoring_db cada uno de esos hechos aparece una sola vez: hay una fila por usuario, identificada con el mismo id que tiene en identity_db.

La repetición existe entre bases, y es la consecuencia elegida de tener una base por servicio (AD-07 y AD-08). Para que no produzca las anomalías que la normalización evita, se cumplen tres condiciones: - Cada dato tiene un único dueño y la copia nunca se edita a mano. - La copia se identifica con la clave del dueño. - El programador la refresca cada cinco minutos (consistencia eventual, aceptada en AD-07).

Gracias a esas copias, la búsqueda de tutores se resuelve sin llamar a otros servicios.

Lo que sí rompía la 3FN era guardar en usuario_ref el nombre de la carrera, porque ahí el mismo hecho se repetía en muchas filas de la misma base. Esa es la diferencia que se corrigió.

Datos derivados que se guardan a propósito

`usuario_ref.reputacion_promedio` y `reputacion_total` se pueden calcular con las evaluaciones visibles del tutor. Dependen solo de la clave (el usuario), así que no rompen la 3FN, pero son redundantes respecto de evaluación. Se guardan porque la búsqueda de tutores ordena por reputación (RNF-02). Se recalculan en la misma transacción que registra o retira una evaluación (RN-06). Los datos de ejemplo cuadran: la reputación guardada de cada tutor coincide con la calculada.

Reglas que la base garantiza sola

Regla	Cómo
RN-02: los bloques de un tutor no se solapan	EXCLUDE USING gist sobre (usuario_id, tstzrange), ignorando los eliminados
RN-03: la primera confirmación gana	índice único parcial <code>ux_reserva_bloque_ocupado</code>
RN-03: nadie reserva su propio bloque	disparador <code>trg_reserva_no_auto</code>
RN-03: una solicitud por estudiante y bloque	índice único parcial <code>ux_reserva_solicitud_unica</code>
RN-05: sin tarifa no hay pago	<code>ck_reserva_pago_con_monto</code> y <code>ck_pago_monto</code>
RN-07: no se confirma sin modalidad ni lugar	<code>ck_reserva_confirmada_completa</code>
RN-08: solo correos @duocuc.cl, sin repetir	<code>ck_usuario_email_dominio</code> y <code>ux_usuario_email</code>

El script `scripts\probar_restricciones.ps1` intenta romper estas reglas en vivo. La base rechaza 19 intentos y acepta uno a propósito: un bloque encima de uno eliminado.

Dos detalles técnicos que conviene tener a mano

- Django 5.2 no escribe ON DELETE en la base: las FK quedan como NO ACTION DEFERRABLE y el CASCADE o RESTRICT lo aplica el ORM. Si alguien borra por SQL dejando huérfanos, la base lo rechaza igual.
- El índice trigram de la búsqueda va sobre `upper(sigla)` y `upper(nombre)`, porque así compara Django y así PostgreSQL puede usarlo.

Diagramas entidad-relación del modelo implementado

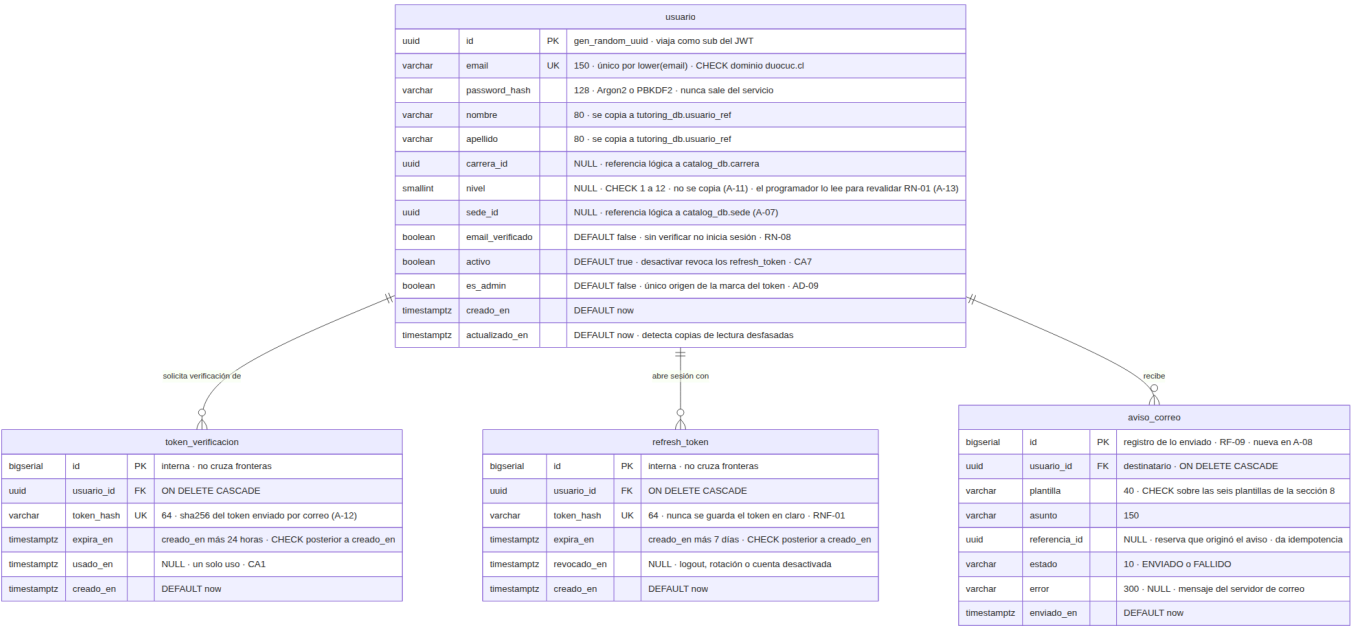


Figura 2. identity_db: 4 tablas.

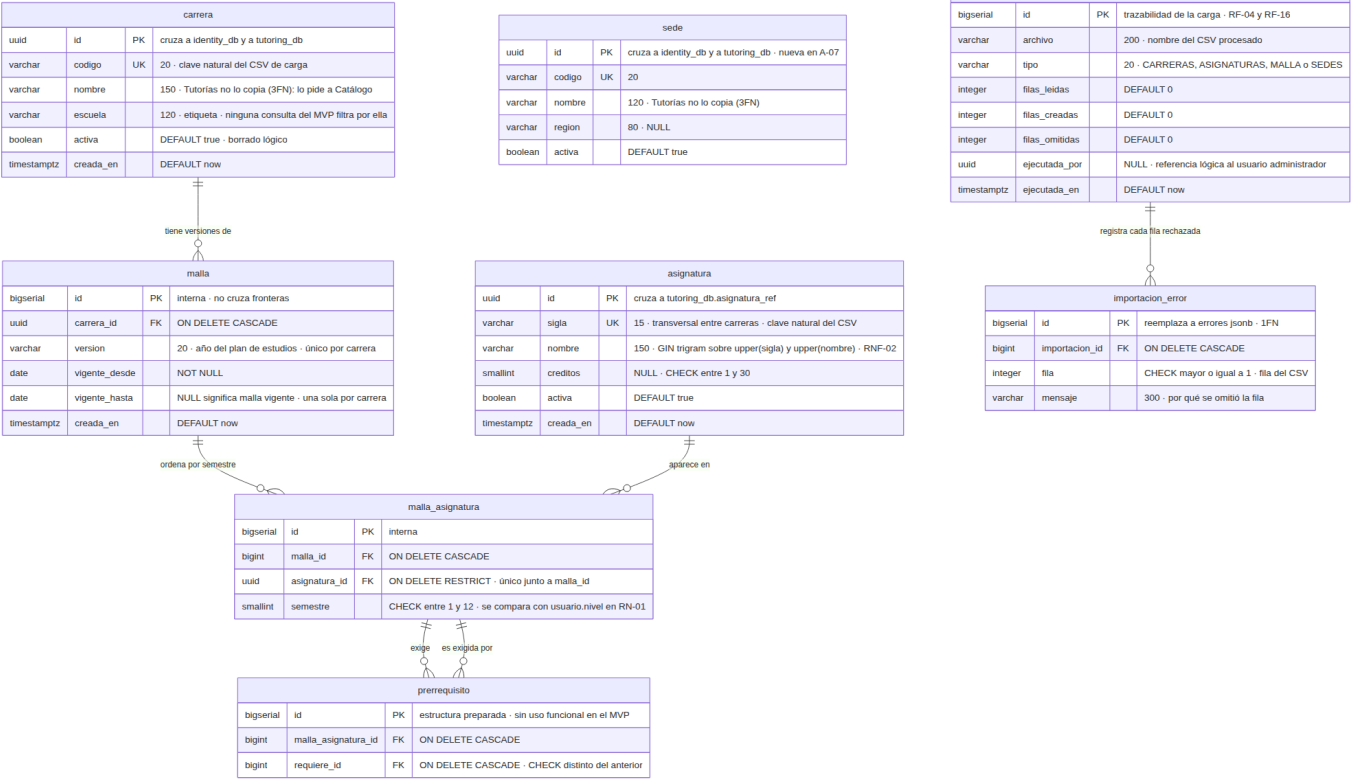


Figura 3. catalog_db: 8 tablas, con importacion_error (1FN).

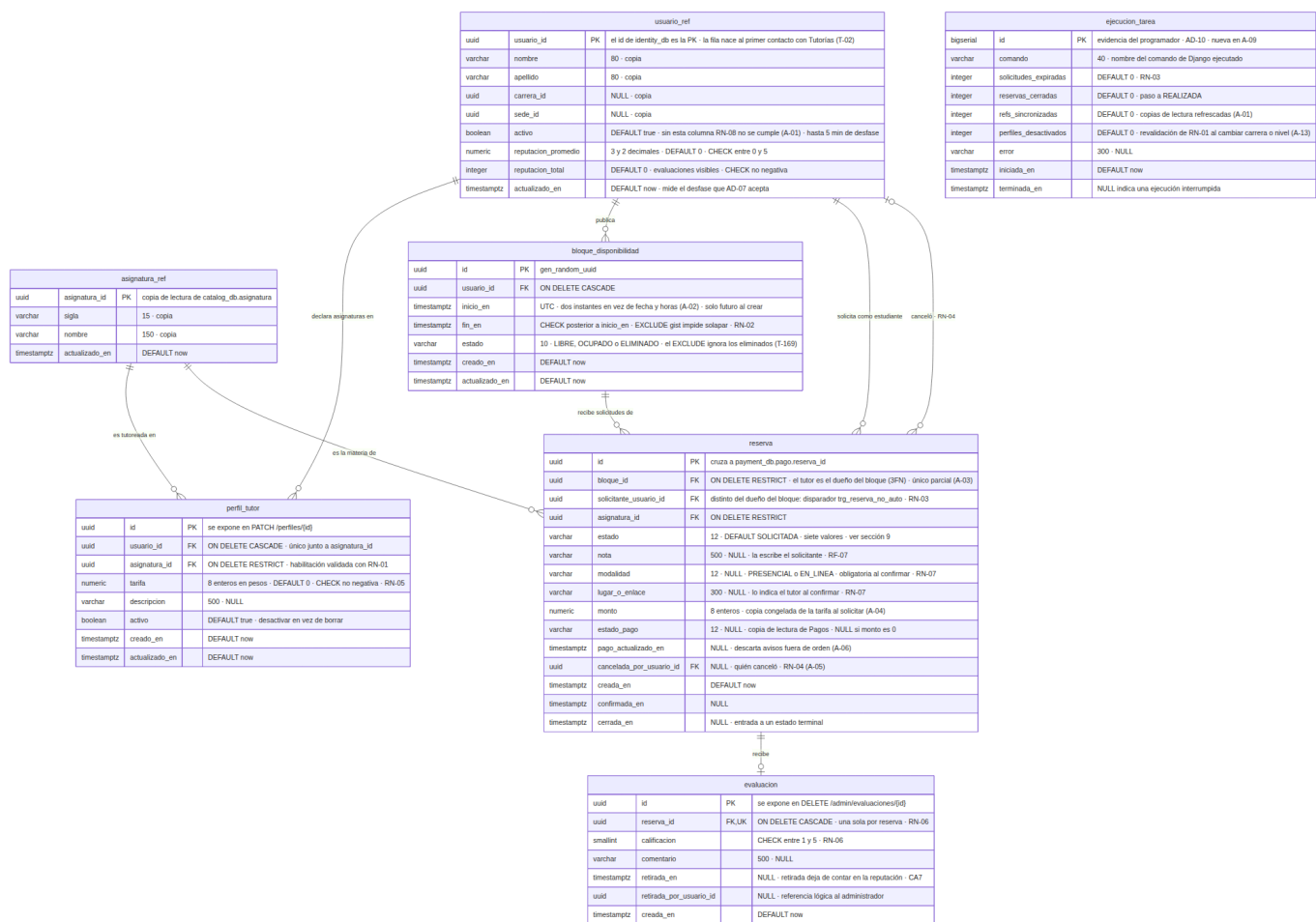


Figura 4. tutoring_db: 7 tablas; reserva llega al tutor por el bloque (3FN).

pago			
uuid	id	PK	gen_random_uuid
uuid	reserva_id	UK	referencia lógica a tutoring_db.reserva · un pago por reserva
numeric	monto		8 enteros en pesos · CHECK mayor que cero · sin tarifa no hay pago
varchar	estado		12 · DEFAULT PENDIENTE · PAGADO, FALLIDO o REEMBOLSADO · RN-05
varchar	proveedor		30 · DEFAULT SIMULADO · punto de integración real · RF-18
varchar	referencia_externa		100 · NULL · única junto a proveedor
timestamptz	notificado_en		NULL · Tutorías ya confirmó el aviso · permite reintentar
timestamptz	creado_en		DEFAULT now
timestamptz	actualizado_en		DEFAULT now



intento_pago			
bigserial	id	PK	historial de reintentos · RN-05 y CA9 · nueva en A-10
uuid	pago_id	FK	ON DELETE CASCADE
varchar	tipo		10 · DEFAULT COBRO · COBRO o REEMBOLSO
varchar	resultado		10 · EXITOSO o FALLIDO
varchar	mensaje		200 · NULL · motivo del rechazo simulado
varchar	referencia_externa		100 · NULL
timestamptz	creado_en		DEFAULT now

Figura 5. payment_db: 2 tablas.